GALAXITECH

# SEE THE SYSTEM

Wardley mapping, one click at a time. DDD Belgium workshop, 20 October 2026.

### The question for tonight

Harbour Insurance gave every product team AI coding agents. Code now arrives in hours, not days. Yet customers get new features no faster than before. Management asks: we pay for AI coding agents, but we don't see faster delivery. What is happening?

## The evening

| TALK | THEN, IN YOUR GROUP |
|---|---|
| 01 Cup of tea | Exercise 1 · Who needs what?<br>Exercise 2 · Build the chain |
| 02 Evolution: the map wakes up | Exercise 3 · Spread it out |
| 03 Tea shop at rush hour | Exercise 4A · Find the constraint<br>Exercise 4B · Draw three pipelines |
| 04 Strategy and the OODA loop<br>Knowledge architecture | Exercise 5 · Simplify and move |
| 05 A real map: EM tooling | Discussion |

## Four people, four roles

| | |
|---|---|
| Domain expert | Knows the company. Has the final word and decides. |
| Facilitator | Starts the talk and makes sure everyone is heard. |
| Timekeeper | Keeps time and keeps the group moving. |
| Clerk | Handles the materials and the canvas, and brings questions to CJ. |

Feel free to swap roles after each exercise.

### The Miro board

Scan to open the board we map on tonight.

miro.com/app/board/uXjVEcI-7Zo=/

### Slides and material

Scan to open the slides, the exercises and this handout.

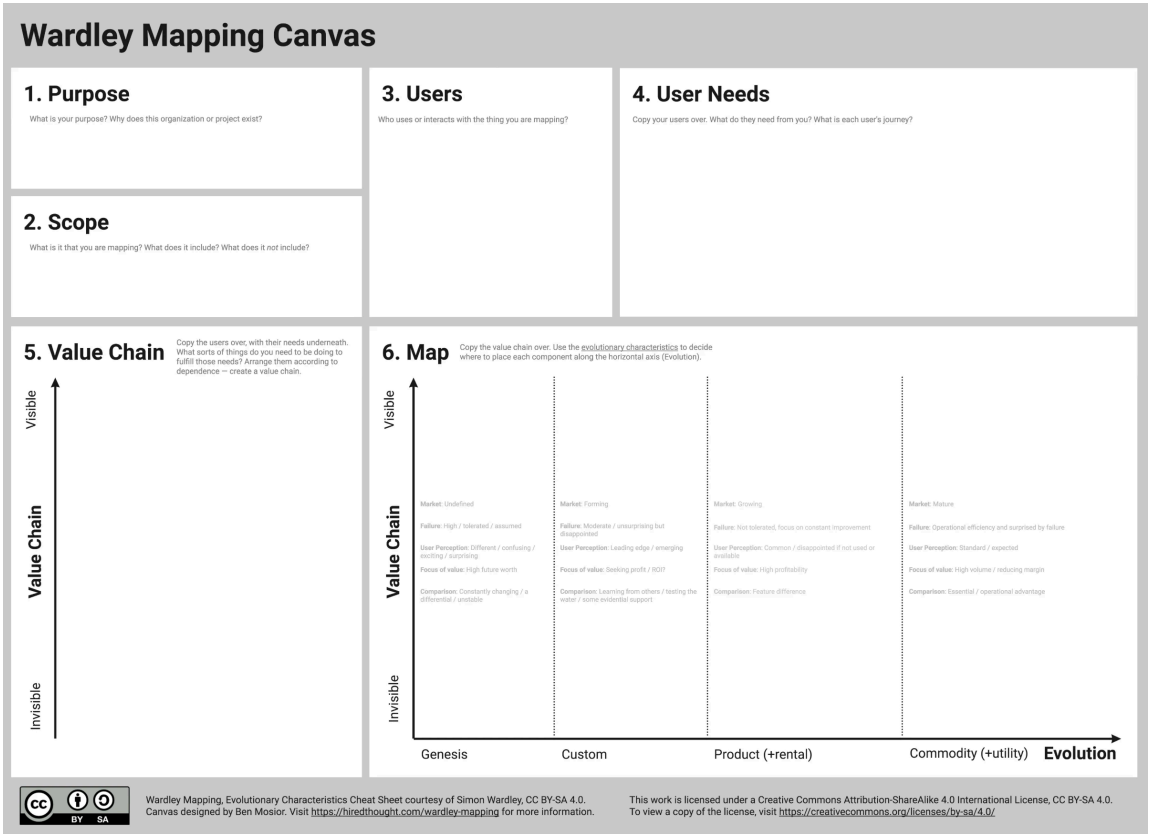
github.com/cjjohansen/DDD-Belgium-Wardley-Mapping-2026

# Harbour Insurance and the six steps

- Harbour Insurance insures ships, cargo and the businesses around the port of Antwerp.
- It has about 2,000 employees and 300,000 policies. Quotes, policies and claims all run through its own software.
- The product and engineering organisation has 6 product teams, about 40 developers and one architect.
- **Purpose:** get new insurance products and changes to customers quickly and safely.
- **Scope for today:** from a product hypothesis to customers using the change.

## Six steps to a Wardley map

| STEP | QUESTION | EXERCISE |
|---|---|---|
| 1 Purpose | Why are we mapping? | 1 |
| 2 Scope | What is in, and what is out? | 1 |
| 3 Users | Who is it for? | 1 |
| 4 User needs | What do they need? | 1 |
| 5 Value chain | What do the needs depend on? | 2 |
| 6 Map | Where does each component sit on the evolution axis? | 3 |




The Wardley Mapping Canvas, designed by Ben Mosior, with the evolution cheat sheet by Simon Wardley. CC BY-SA 4.0. hiredthought.com/wardley-mapping

# Exercise 1: who needs what?

Tonight we map an internal value chain, not the insurance marketplace: how Harbour Insurance delivers change. Its users are the people who make change happen. Write the purpose and the scope first. Then name the users of the delivery system and, under each user, their needs in their own words. Stop there: the value chain comes in Exercise 2.

1 PURPOSE

2 SCOPE: IN AND OUT

3 USER

4 THEIR NEEDS

3 USER

4 THEIR NEEDS

3 USER

4 THEIR NEEDS

3 USER

4 THEIR NEEDS

3 USER

4 THEIR NEEDS

3 USER

4 THEIR NEEDS

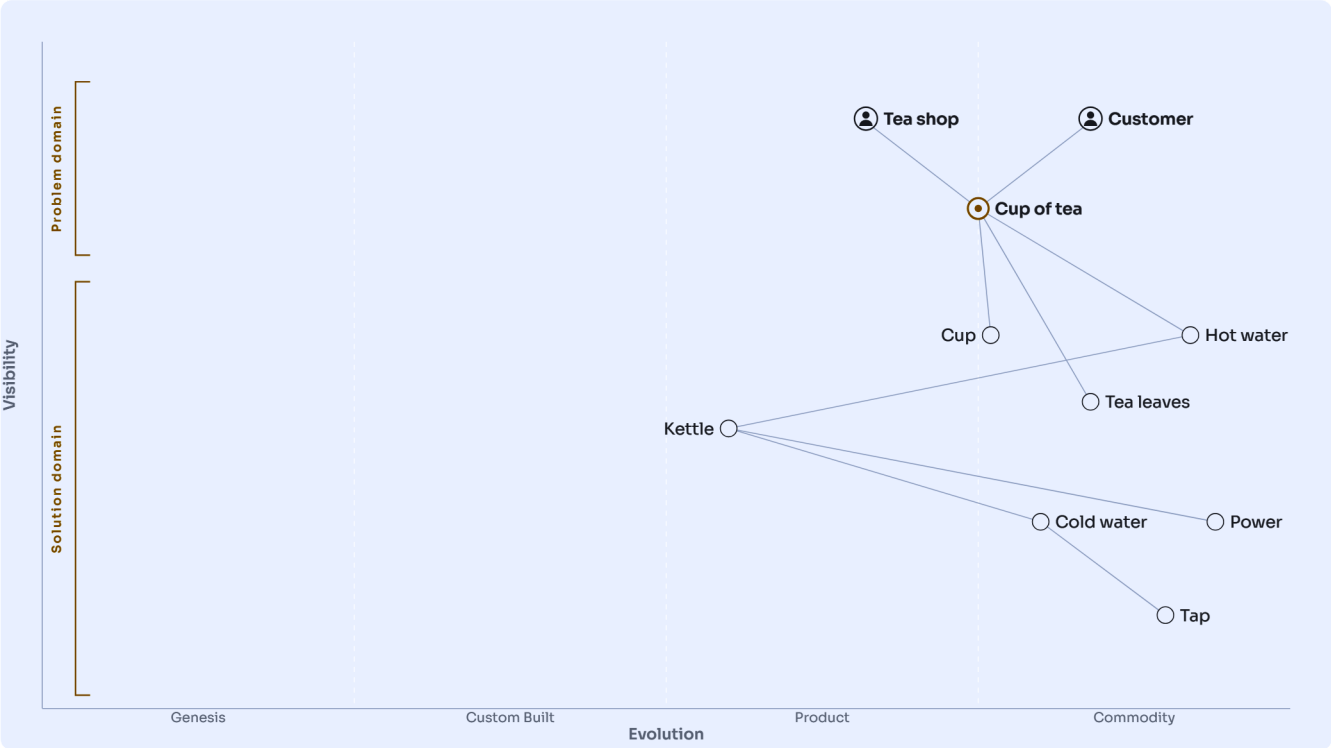
### Before Exercise 2: pick your anchor

To build one map, you need one anchor. Ask what question the map answers. For Harbour Insurance it is "Why is delivery slow?" Pick the user whose need matters for that question. The other needs become components in your chain.

THE QUESTION THIS MAP ANSWERS

ANCHOR USER AND THEIR NEED

# How to read a Wardley map




The cup of tea: a customer needs a cup of tea, which needs tea leaves, hot water, a kettle and power.

## Reading the value chain

- Components near the top are visible to the user. Components near the bottom are hidden from the user.
- Read down to ask how: how is this need met?
- Read up to ask why: why do we need this?
- Each link is a dependency. The component above pays money or effort, and gets a capability back.
- A map is a chain of needs.

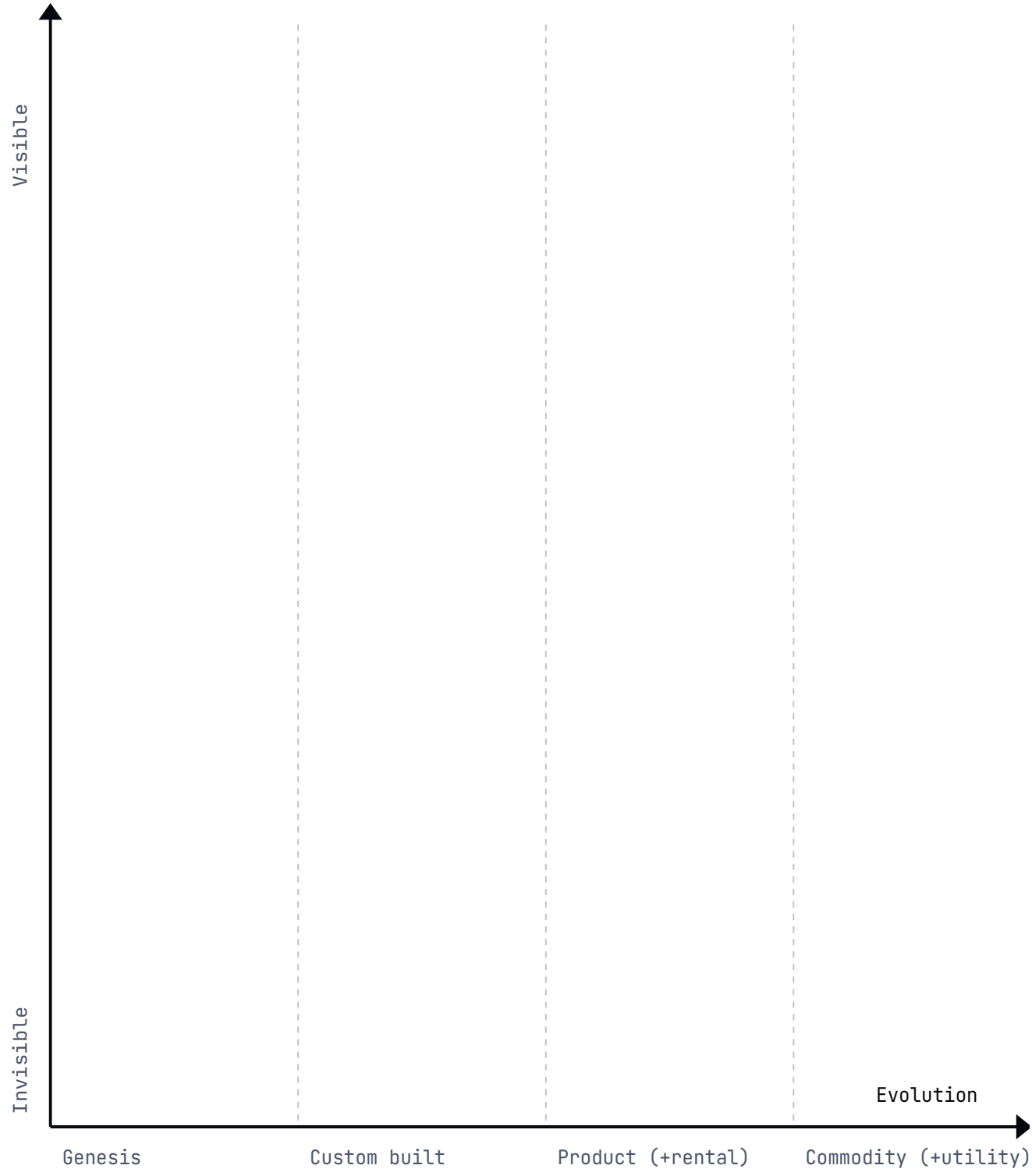
## What goes on a map

- Activities: what we do. Making a cup of tea.
- Practices: how we do it. Hand brewing.
- Data: facts we keep. The menu and the prices.
- Knowledge: what we understand. How strong tea should be.
- Things we use: the cup, the kettle.

# Exercises 2 and 3: build the chain, then spread it out

**Exercise 2.** Start from your anchor and its need. Ask "what does this depend on?" until you reach infrastructure. Stack the components by visibility, most visible at the top. Do not place them on the evolution axis yet.

**Exercise 3.** Take your chain and place every component on the evolution axis: genesis, custom built, product, commodity. Users and needs stay at the top. If you are stuck on a placement, the domain expert decides.



# Everything evolves, and the way we work should follow

Every component evolves from left to right through supply and demand competition. You cannot stop it. You can only speed it up or slow it down.

## Evolution: four sets of stage labels

| | I | II | III | IV |
|---|---|---|---|---|
| Activities | Genesis | Custom built | Product (+rental) | Commodity (+utility) |
| Practices | Novel | Emerging | Good | Best |
| Data | Unmodelled | Divergent | Convergent | Modelled |
| Knowledge | Concept | Hypothesis | Theory | Accepted |

Practices evolve with the activity. When servers became a utility, autoscaling replaced capacity planning. Code is now close to a commodity, so code review will have to evolve too.

## Doctrine: use appropriate methods

One way of working does not fit the whole map. Pick the way of working by the stage each component sits in, not by habit.

| | GENESIS, CUSTOM BUILT | PRODUCT | COMMODITY, UTILITY |
|---|---|---|---|
| The goal | Learn fast | Get it right | Cut cost and variation |
| How we work | Agile: small experiments | Lean: improve the flow | Six Sigma: standardise and automate |
| How we get it | Build it ourselves | Buy it and fit it in | Rent it as a utility |
| At Harbour Insurance | The product hypothesis | Event model tooling | AI coding agents, CI |

After Simon Wardley, Wardley Maps, CC BY-SA 4.0.

# Where work waits, and the several ways to do one thing

## Constraint and inertia

- A constraint is a capacity limit. Work piles up in front of it.
- Inertia is when the old way is kept although a better one exists.
- To find inertia, ask who would lose if the component moved.

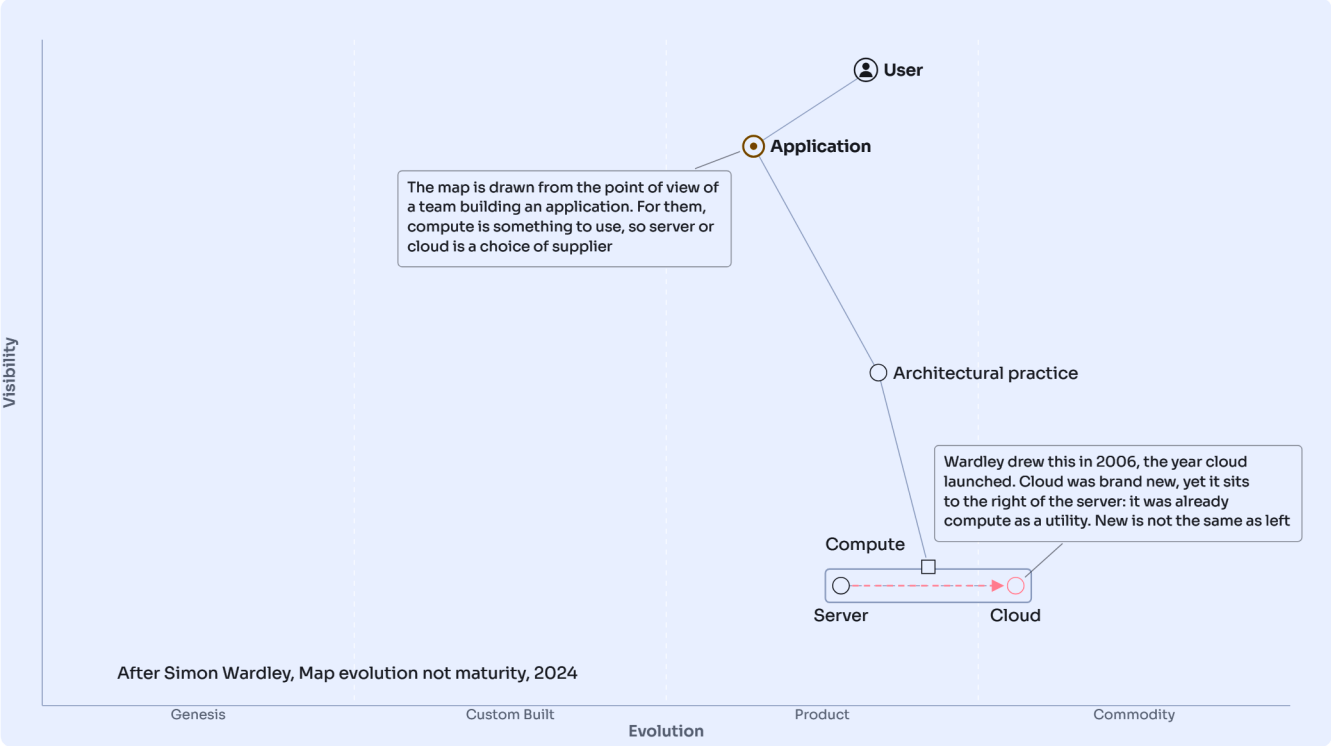
## Goldratt's five steps

1. Identify the constraint.
2. Exploit it: get the most out of what is already there.
3. Subordinate everything else to it.
4. Elevate it: add capacity, and only now.
5. Repeat: the constraint has moved.

### A pipeline, in Simon Wardley's sense

A pipeline is one component that is done in several ways at the same time, each way at its own stage of evolution. You draw it as a box along the evolution axis with each way inside it. A new invention starts on the left. The industrialised form of an old activity goes on the right, however new it is. If people move from one way to another, draw an arrow.

**This is not a delivery pipeline.** Build, test and release are steps that work passes through. A Wardley pipeline holds different ways of doing the same thing.




Compute as a pipeline, redrawn after Simon Wardley's "Map evolution not maturity": server and cloud side by side, with people moving from server to cloud.

# Exercise 4: find the constraint, draw three pipelines

## 4A · Find the constraint

THE CONSTRAINT: WHERE DOES WORK WAIT LONGEST?

ONE INERTIA: WHAT RESISTS CHANGE, AND WHO WOULD LOSE IF IT MOVED?

| EXPLOIT, WITH NO BUDGET | SUBORDINATE THE REST | ELEVATE: WHAT TO INVEST IN |
|---|---|---|
| | | |
| | | |
| | | |

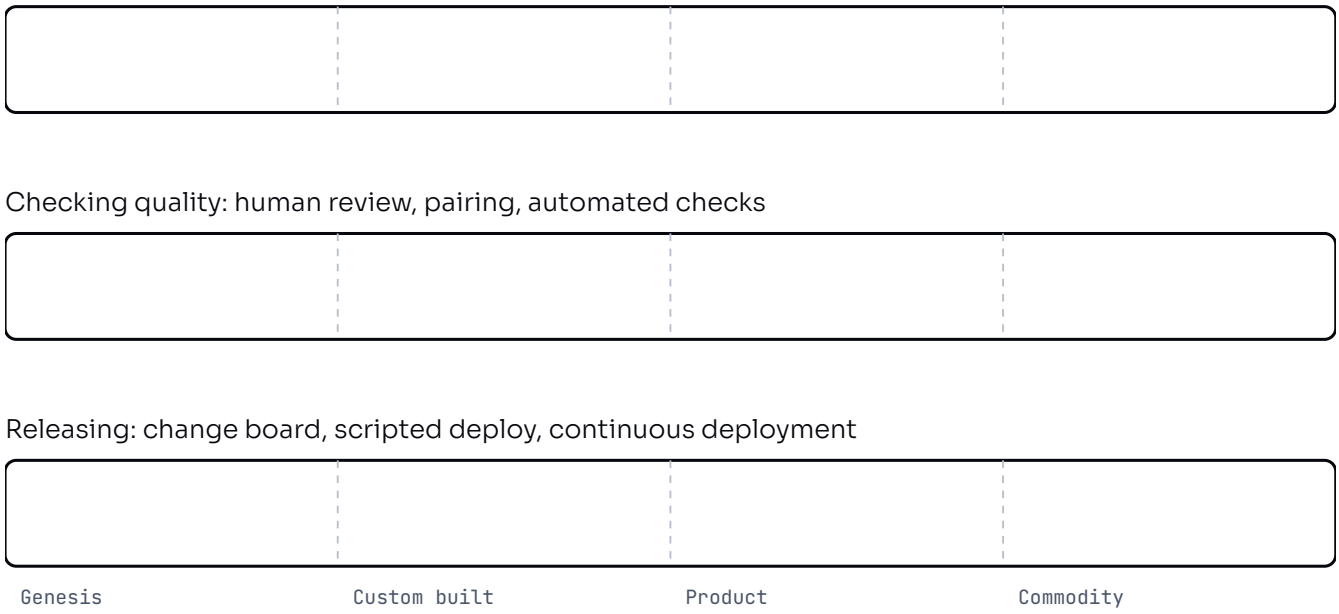
## 4B · Draw three pipelines

Place each way at its own stage of evolution. Then ask: which pipeline moved right, and which one holds the constraint?

Writing code: by hand, with an AI assistant, by AI agents

Checking quality: human review, pairing, automated checks

Releasing: change board, scripted deploy, continuous deployment



# Exercise 5: simplify and move

### First: cut it down

Write the question your map answers. Remove what does not change the answer. Aim for 12 components or fewer.

### Four ways to make a map readable

- One question per map: keep what changes the answer.
- One map per user, when their needs really differ.
- Map around the constraint: the chain to it, nothing else.
- Fold a cluster into a submap.

### Then: make a move

Draw one strategic move on your simplified map. Use a dashed arrow for natural evolution and a solid arrow for a move you intend. Pick a gameplay pattern that speeds it up or holds it back.

### Gameplay: moves you can choose

- Open approaches: commoditise a layer someone else owns.
- Innovate, leverage, commoditise.
- Standards game: set the rules others build on.
- Exploit inertia: move where a rival cannot follow.
- Co-creation: build it with users, not for them.

### Last: the travelling map

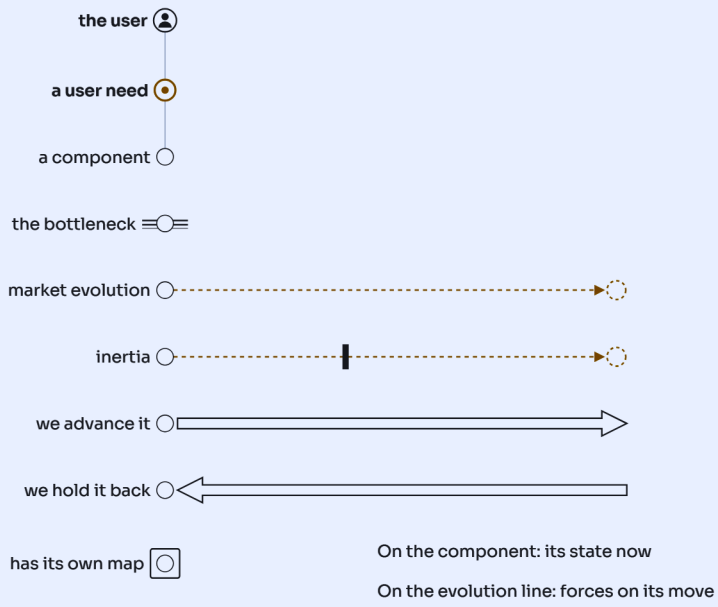
Two of you take your map to the next group (group 6 goes to group 1). Explain your question, your map and your move. The hosts ask what you left out and whether you would dare that move. Then swap. About 10 minutes.

THE QUESTION OUR MAP ANSWERS

WHAT WE CUT, AND WHY

OUR MOVE, THE PATTERN WE CHOSE, WHO GAINS AND WHO LOSES

# The legend, and where to read on




The marks used on the maps tonight.

## If you want to read on

- **Simon Wardley, Wardley Maps.** Start here. Free to read at medium.com/wardleymaps and on Leanpub.
- Susanne Kaiser, Architecture for Flow, and Nick Tune, Architecture Modernization. Both write for a DDD audience.
- David Anderson, Mark McCann and Michael O'Reilly, The Value Flywheel Effect.
- Will Larson, Crafting Engineering Strategy. James Urquhart, Flow Architectures. Gregor Hohpe, Platform Strategy.
- Erik Schön, The Art of Strategy. Its free annex collects 168 Wardley maps from 90 mappers.
- Joaquín Peña Fernández, Crossing the Chasm with Wardley Maps. E. Alex Hudson, Practical Introduction to Wardley Mapping.

### Slides and material

github.com/cjjohansen/DDD-Belgium-Wardley-Mapping-2026
galaxi.tech · cj@galaxi.tech · linkedin.com/in/cjjohansen

Wardley mapping material after Simon Wardley, CC BY-SA 4.0. Canvas by Ben Mosior, CC BY-SA 4.0.